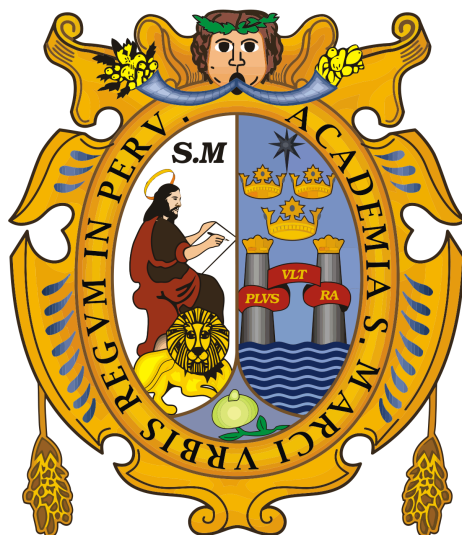


«Año de la Esperanza y el Fortalecimiento de la Democracia»

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



CURSO:

Redes de Computadora

DOCENTE:

Hernan Oswaldo Villafuerte Barreto

ENTREGABLE:

4

INTEGRANTES:

Carlos Nicolas Morales Cuellar (23190329)

Ramos Calzada Jose Martin (23190341)

Ronald Enrique Rojas Crisostomo (23190346)

Perú - Lima

2026

1. Comparación con otras cámaras del mercado

Aunque una cámara comercial está empaquetada para el usuario final, nuestra arquitectura ofrece ventajas técnicas inmensas desde la perspectiva de la ingeniería, el control de redes y la ciberseguridad.

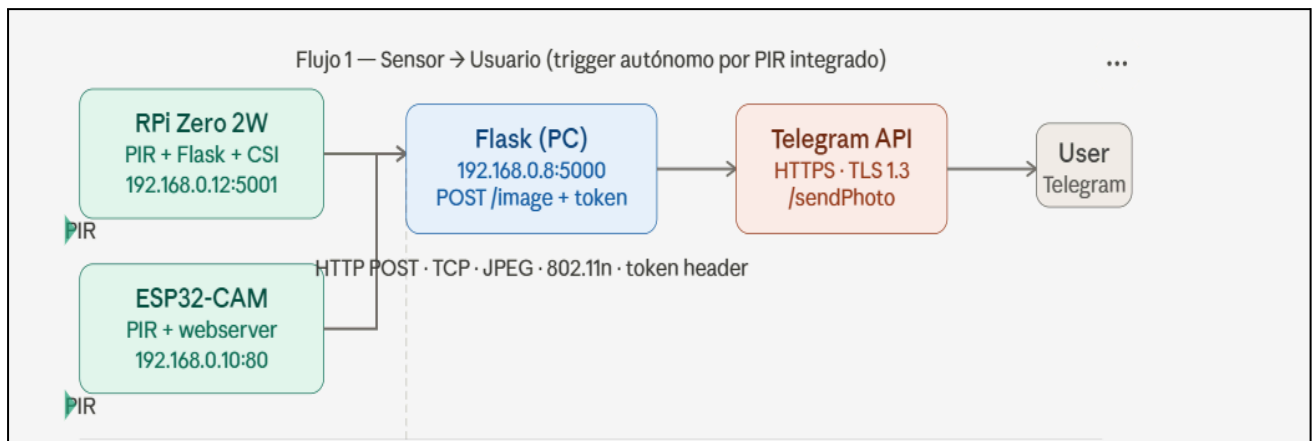
- **Transparencia y Observabilidad Total (Caja Blanca vs. Caja Negra):** En el ámbito de las telecomunicaciones y la seguridad de la red, una cámara comercial funciona como una "caja negra". El usuario final no tiene visibilidad de qué metadatos se exportan, a qué puertos se conectan los dispositivos internamente o con qué frecuencia realizan telemetría. La arquitectura propuesta soluciona esta falta de observabilidad. Al diseñar el flujo desde cero, se permite una inspección profunda (DPI) con herramientas como Wireshark, lo que garantiza que no exista tráfico oculto y permite auditar cabeceras, tiempos de respuesta y secuencias TCP de forma precisa.
- **Soberanía de los Datos y Edge Computing:** Las soluciones comerciales convencionales suelen enviar un stream constante de video a servidores de terceros (muchas veces alojados en otros continentes), lo que satura el ancho de banda de subida (uplink) de la red local. Este proyecto implementa un enfoque de Edge Computing: el procesamiento y la decisión inicial ocurren localmente gracias a los sensores PIR y a la compresión JPEG mediante hardware/CPU. El sistema solo consume los 2 a 5 Mbps de throughput requeridos cuando existe un trigger físico o un comando explícito, optimizando drásticamente la eficiencia energética y el tráfico del enrutador.
- **Desacoplamiento de Servicios (Middleware Local):** Las cámaras de mercado obligan al usuario a depender de su aplicación móvil específica. En este proyecto, el servidor Flask actúa como un dispatcher intermediario. Esta es una ventaja arquitectónica gigante: los sensores (ESP32/RPi) nunca están expuestos directamente a Internet. Flask traduce las peticiones locales a tráfico HTTPS seguro hacia Telegram. Si en el futuro se desea cambiar Telegram por una alerta por correo, un servidor web propio o integrarlo con un SIEM, solo se debe modificar el código del dispatcher local sin tocar el firmware de las cámaras.

- **Flexibilidad en Ciberseguridad y Hardening:** Un sistema cerrado limita las pruebas de penetración o la reestructuración de la red sin violar los términos de servicio. Con este sistema IoT, es posible aplicar medidas de seguridad de grado industrial, como la segmentación lógica del tráfico mediante VLANs (IEEE 802.1Q) para aislar los nodos de percepción. Conocer exactamente qué puertos deben estar en estado listening (ej. 5000, 5001, 80) permite crear reglas de firewall estrictas y personalizadas, lo cual es el estándar ideal para entornos de misión crítica.

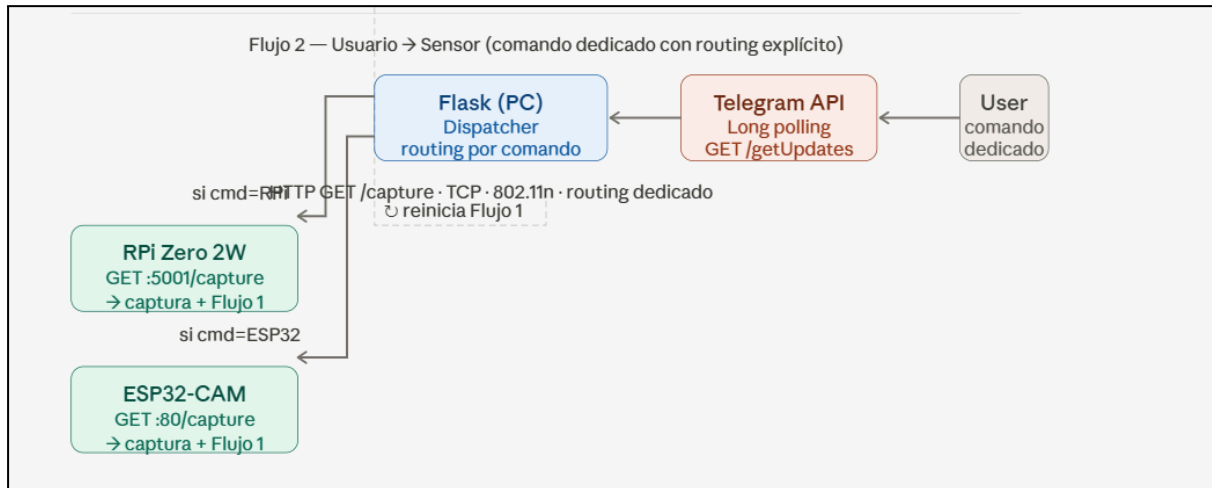
2. Integración de Tecnología Emergente

La arquitectura del sistema se fundamenta en la integración de dos paradigmas tecnológicos clave: Internet de las Cosas (IoT) y Edge Computing.

- **Implementación en el Borde (Edge Computing):** En lugar de enviar datos crudos a un servidor centralizado para su análisis, el procesamiento primario ocurre de forma local en los nodos sensores (Raspberry Pi Zero 2W y ESP32-CAM). Estos dispositivos de recursos limitados se encargan de gestionar los triggers físicos (sensores PIR), capturar las imágenes y realizar la compresión JPEG mediante hardware o CPU local antes de transmitirlos.



- **Interoperabilidad IoT - Nube:** El sistema emplea un servidor Flask local que actúa como dispatcher (proxy o enrutador de comandos). Esta capa intermedia consolida el tráfico de la red LAN y gestiona la comunicación asíncrona hacia la nube mediante la API de Telegram, logrando un sistema bidireccional eficiente y con baja latencia.



3. Análisis de Protocolos de Red

El flujo de comunicación del proyecto ha sido validado empíricamente utilizando herramientas de análisis como Wireshark y Nmap, mapeando el tráfico en las diferentes capas del modelo TCP/IP:

Capa de Aplicación (L7)

- **HTTP/1.1:** Se utiliza en la red local (LAN) para la comunicación entre los nodos sensores y el servidor Flask. Las imágenes se transmiten utilizando solicitudes POST con formato multipart/form-data, y se emplean Tokens Bearer para autenticación.
- **HTTPS (TLS 1.3):** El servidor Flask traduce las peticiones locales a tráfico seguro para comunicarse con la API de Telegram a través de Internet. El cifrado TLS protege los metadatos y credenciales (tokens) en la red pública, manteniendo un esquema de Long Polling basado en datos JSON.

```
C:\Windows\system32>nmap -Pn -sT -sV -p- 192.168.0.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 21:24 -0500
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 75.11% done; ETC: 21:24 (0:00:05 remaining)
Stats: 0:00:23 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 21:24 (0:00:00 remaining)
Nmap scan report for 192.168.0.12
Host is up (0.0056s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 10.0p2 Raspbian 7+deb13u2 (protocol 2.0)
5001/tcp  open  http     Werkzeug httpd 3.1.8 (Python 3.13.5)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 39.56 seconds
```

Figura 1. Escaneo de puertos demostrando el servicio HTTP (Flask) activo en la capa de aplicación.

Capa de Transporte (L4)

- **TCP:** El sistema depende exclusivamente del protocolo TCP para asegurar la entrega ordenada de los payloads (comandos e imágenes). Mediante escaneos, se corroboró el estado open (bandera syn-ack) en los puertos locales (80 para ESP32, 5001 para Flask), lo que confirma la ejecución exitosa del three-way handshake. Se generan sesiones efímeras para capturas locales y sesiones semi-persistentes para la consulta a Telegram.

(Windows\system32)netstat -ano

Conexiones activas

Proto	Dirección local	Dirección remota	Estado	PID
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	424
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:3580	0.0.0.0:0	LISTENING	3956
TCP	0.0.0.0:5000	0.0.0.0:0	LISTENING	9592
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	3796
TCP	0.0.0.0:5357	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:7680	0.0.0.0:0	LISTENING	3548
TCP	0.0.0.0:40664	0.0.0.0:0	LISTENING	808
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING	720
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING	1300
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	1612
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING	3192
TCP	0.0.0.0:49671	0.0.0.0:0	LISTENING	792
TCP	0.0.0.0:50065	0.0.0.0:0	LISTENING	4996
TCP	0.0.0.0:50110	0.0.0.0:0	LISTENING	5144
TCP	0.0.0.0:50111	0.0.0.0:0	LISTENING	5228
TCP	127.0.0.1:2680	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:5939	0.0.0.0:0	LISTENING	3480
TCP	127.0.0.1:27182	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:49672	127.0.0.1:49673	ESTABLISHED	5228
TCP	127.0.0.1:49673	127.0.0.1:49672	ESTABLISHED	5228
TCP	127.0.0.1:49674	127.0.0.1:49675	ESTABLISHED	5136
TCP	127.0.0.1:49675	127.0.0.1:49674	ESTABLISHED	5136
TCP	127.0.0.1:49676	127.0.0.1:49677	ESTABLISHED	5144
TCP	127.0.0.1:49677	127.0.0.1:49676	ESTABLISHED	5144
TCP	127.0.0.1:49678	0.0.0.0:0	LISTENING	5124
TCP	127.0.0.1:49678	127.0.0.1:49683	ESTABLISHED	5124
TCP	127.0.0.1:49683	127.0.0.1:49678	ESTABLISHED	3740
TCP	127.0.0.1:49789	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:50034	0.0.0.0:0	LISTENING	4996
TCP	127.0.0.1:50034	127.0.0.1:50048	ESTABLISHED	4996
TCP	127.0.0.1:50035	0.0.0.0:0	LISTENING	12628
TCP	127.0.0.1:50035	127.0.0.1:50036	ESTABLISHED	12628
TCP	127.0.0.1:50036	127.0.0.1:50035	ESTABLISHED	4996
TCP	127.0.0.1:50048	127.0.0.1:50034	ESTABLISHED	4704
TCP	127.0.0.1:50067	127.0.0.1:50068	ESTABLISHED	12628
TCP	127.0.0.1:50068	127.0.0.1:50067	ESTABLISHED	1544
TCP	127.0.0.1:50237	127.0.0.1:5000	TIME_WAIT	0
TCP	127.0.0.1:55878	0.0.0.0:0	LISTENING	10636
TCP	192.168.0.8:139	0.0.0.0:0	LISTENING	4
TCP	192.168.0.8:49788	2.23.232.59:443	CLOSE_WAIT	13516
TCP	192.168.0.8:50024	192.168.0.12:22	ESTABLISHED	3620
TCP	192.168.0.8:50072	149.154.175.53:443	ESTABLISHED	5224
TCP	192.168.0.8:50236	149.154.166.110:443	TIME_WAIT	0
TCP	192.168.0.8:50240	149.154.175.52:443	ESTABLISHED	5224
TCP	192.168.0.8:50242	149.154.175.53:80	TIME_WAIT	0
TCP	192.168.0.8:50243	149.154.175.52:80	TIME_WAIT	0
TCP	192.168.56.1:139	0.0.0.0:0	LISTENING	4

Figura 2. Tabla de conexiones TCP activas en el nodo despachador validando las sesiones establecidas.

Capa de Red / Internet (L3)

- **IPv4:** El direccionamiento opera bajo el estándar RFC 1918 en el segmento de subred plano 192.168.0.0/24.
- **DHCP:** Las direcciones IP de los nodos (ej. 192.168.0.8 para el dispatcher y 192.168.0.12 para la Raspberry Pi) son asignadas de forma dinámica y gestionadas por el servidor DHCP del gateway, optimizando la escalabilidad local.
- **ICMP:** Empleado en pruebas de capa 3 para el descubrimiento de hosts activos (Echo Request/Reply) y la evaluación de disponibilidad, arrojando 0% de pérdida de paquetes y latencias promediadas en LAN de 3.5 ms (ESP32) y 38 ms (RPi)

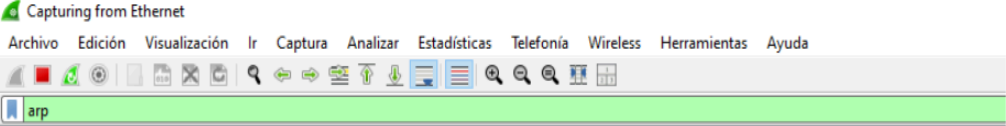
```
PS C:\Users\PC - Usuario> for ($i = $start; $i -le $end; $i++) {
>>     $ip = "$network.$i"
>>
>>     # Enviar 2 solicitudes ICMP
>>     $ping = Test-Connection -ComputerName $ip -Count 2 -ErrorAction SilentlyContinue
>>
>>     # Filtrar solo respuestas exitosas (Echo Reply)
>>     $success = $ping | Where-Object { $_.StatusCode -eq 0 }
>>
>>     if ($success -and $success.Count -gt 0) {
>>         # Métricas
>>         $avg = ($success | Measure-Object -Property ResponseTime -Average).Average
>>         $min = ($success | Measure-Object -Property ResponseTime -Minimum).Minimum
>>         $max = ($success | Measure-Object -Property ResponseTime -Maximum).Maximum
>>         $ttl = $success[0].TimeToLive
>>
>>         # Salida en consola
>>         Write-Host ("[SUCCESS] {0} | Respuestas: {1}/2 | Prom: {2} ms | Min: {3} ms | Max: {4} ms | TTL: {5}" -f `
>>             $ip,
>>             $success.Count,
>>             [math]::Round($avg,2),
>>             $min,
>>             $max,
>>             $ttl
>>         )
>>     }
>> }
[SUCCESS] 192.168.0.1 | Respuestas: 2/2 | Prom: 0.5 ms | Min: 0 ms | Max: 1 ms | TTL: 80
[SUCCESS] 192.168.0.3 | Respuestas: 2/2 | Prom: 1.5 ms | Min: 1 ms | Max: 2 ms | TTL: 80
[SUCCESS] 192.168.0.5 | Respuestas: 2/2 | Prom: 35.5 ms | Min: 3 ms | Max: 68 ms | TTL: 80
[SUCCESS] 192.168.0.6 | Respuestas: 2/2 | Prom: 47 ms | Min: 16 ms | Max: 78 ms | TTL: 80
[SUCCESS] 192.168.0.7 | Respuestas: 2/2 | Prom: 514 ms | Min: 494 ms | Max: 534 ms | TTL: 80
[SUCCESS] 192.168.0.8 | Respuestas: 2/2 | Prom: 0 ms | Min: 0 ms | Max: 0 ms | TTL: 80
[SUCCESS] 192.168.0.9 | Respuestas: 2/2 | Prom: 255.5 ms | Min: 204 ms | Max: 307 ms | TTL: 80
[SUCCESS] 192.168.0.10 | Respuestas: 2/2 | Prom: 3.5 ms | Min: 2 ms | Max: 5 ms | TTL: 80
[SUCCESS] 192.168.0.11 | Respuestas: 2/2 | Prom: 49 ms | Min: 22 ms | Max: 76 ms | TTL: 80
[SUCCESS] 192.168.0.12 | Respuestas: 2/2 | Prom: 38 ms | Min: 3 ms | Max: 73 ms | TTL: 80
```

Figura 3. Ejecución de script de descubrimiento ICMP demostrando la disponibilidad y métricas de latencia (RTT) en la red local.

Capa de Acceso a la Red / Enlace (L1-L2)

- **IEEE 802.11n:** La conectividad de los nodos IoT se gestiona a través de la banda inalámbrica de 2.4 GHz (Wi-Fi 4), empleando el mecanismo de control de acceso al medio CSMA/CA para prevenir colisiones.
- **ARP:** Se evidenció el correcto funcionamiento del protocolo de resolución de direcciones (ARP Announcement y Requests), vital para asociar IPs lógicas

con las direcciones MAC físicas (ej. interfaces Espressif y Raspberry Pi Trading) y permitir la transferencia de tramas Ethernet sobre la WLAN.



Capturing from Ethernet

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

arp

No.	Time	Source	Destination	Protocol	Length	Info
71	1.800371800	0a:e0:af:dc:0a:bf	Broadcast	ARP		42 Who has 192.168.0.12? Tell 192.168.0.8
72	1.882177900	RaspberryPi_5a:fc:5e	0a:e0:af:dc:0a:bf	ARP		60 192.168.0.12 is at 88:a2:9e:5a:fc:5e
207	6.354767900	Espressif_05:0c:7c	Broadcast	ARP		60 ARP Announcement for 192.168.0.10
889	44.536550800	0a:e0:af:dc:0a:bf	SagemcomBroa_96:f6:...	ARP		42 Who has 192.168.0.1? Tell 192.168.0.8
890	44.537028700	SagemcomBroa_96:f6:...	0a:e0:af:dc:0a:bf	ARP		60 192.168.0.1 is at a0:39:ee:96:f6:fc

Figura 3. Captura de tráfico ARP en Wireshark evidenciando la resolución de direcciones MAC e identificación de fabricantes en los nodos IoT.